NAMA : NAYLA RABIA GUSTARI
NPM : 20241320034
MATKUL : PEMOGRAMAN MODERN (PYTHON)

A. SOAL TEORI

1. *Jelaskan perbedaan antara syntax error dan exception dalam Python. Berikan masing-masing dua contoh!*

- Syntax error adalah kesalahan penulisan kode yang tidak sesuai dengan aturan sintaks Python. Kesalahan ini harus diperbaiki sebelum program dapat dijalankan.

  Contoh:

- Kurang tanda kurung penutup pada perintah `print()`.
- Kesalahan indentasi pada blok `if`, `for`, atau fungsi.

  Exception adalah kesalahan yang terjadi ketika program sedang berjalan (runtime), meskipun penulisan sintaksnya sudah benar.

  Contoh:

- `ZeroDivisionError`: melakukan pembagian dengan angka nol.
- `IndexError`: mengakses indeks list yang berada di luar jangkauan.

  Jadi, perbedaannya adalah syntax error terjadi karena kesalahan penulisan kode, sedangkan exception terjadi saat program dijalankan.

2. *Apa perbedaan fungsi blok else dan blok finally dalam struktur tryexcept-else-finally? Kapan masing-masing dijalankan?*

- **Blok else** dijalankan hanya jika tidak terjadi exception pada blok try. Blok ini digunakan untuk menjalankan proses lanjutan ketika operasi berhasil.

  Sedangkan **blok finally** selalu dijalankan, baik terjadi exception maupun tidak. Blok ini biasanya digunakan untuk membersihkan sumber daya, seperti menutup koneksi database atau file. Contohnya dalam Sistem Informasi Akademik:

- Else: menampilkan data mahasiswa jika pengambilan data dari database berhasil.
- Finally: menutup koneksi database setelah proses selesai, baik berhasil maupun gagal.

3. *Mengapa urutan blok except penting? Apa yang terjadi jika exception yang lebih umum ditulis sebelum exception yang lebih spesifik?*

- Urutan blok except penting karena Python akan memeriksa exception berdasarkan urutan penulisannya. Exception yang lebih spesifik harus ditulis terlebih dahulu, kemudian diikuti exception yang lebih umum.

  Jika exception umum seperti `Exception` ditulis lebih dahulu, exception tersebut akan menangkap semua exception turunannya. Akibatnya, blok except yang lebih

spesifik tidak akan mendapatkan kesempatan untuk dijalankan. Oleh karena itu, urutan yang benar adalah menangani exception spesifik terlebih dahulu, kemudian exception umum.

4. *Jelaskan keuntungan membuat hierarki custom exception dibanding menggunakan exception bawaan Python secara langsung. Berikan contoh dalam konteks Sistem Informasi Akademik!*

- Hierarki custom exception adalah pembuatan kelas exception sendiri yang disusun berdasarkan hubungan pewarisan (inheritance).

  Keuntungannya:

1. Memudahkan pengelompokan kesalahan berdasarkan kebutuhan aplikasi.
2. Membuat pesan kesalahan lebih spesifik dan mudah dipahami.
3. Memudahkan penanganan kesalahan pada berbagai tingkat program.
4. Memisahkan kesalahan berdasarkan masalah bisnis, bukan hanya masalah teknis.

  Contoh dalam Sistem Informasi Akademik:

- `SIAkademikError`: exception utama untuk seluruh kesalahan sistem akademik.
- `MahasiswaError`: kesalahan yang berkaitan dengan data mahasiswa.
- `NIMFormatError`: format NIM tidak sesuai.
- `NIMDuplikatError`: NIM sudah terdaftar.
- `MatakuliahError`: kesalahan yang berkaitan dengan mata kuliah.

  Dengan hierarki tersebut, sistem dapat menangani kesalahan secara khusus maupun secara umum melalui kelas induknya.

5. *Jelaskan perbedaan antara debugging menggunakan print statement dan debugging menggunakan pdb. Dalam situasi apa masing-masing lebih tepat digunakan?*

- Print statement merupakan teknik debugging sederhana dengan menambahkan perintah print untuk menampilkan nilai variabel dan memeriksa alur program.

  Kelebihan:

- Mudah digunakan.
- Cepat untuk memeriksa nilai variabel.
- Cocok untuk menemukan kesalahan sederhana.

  pdb (Python Debugger) merupakan debugger bawaan Python yang memungkinkan pengembang menghentikan program pada titik tertentu dan memeriksa kondisi program secara interaktif.

  Kelebihan:

- Dapat menggunakan breakpoint.

- Memeriksa nilai variabel secara langsung.
- Menjalankan kode langkah demi langkah.
- Menelusuri alur pemanggilan fungsi.

  Print statement lebih tepat digunakan untuk kesalahan sederhana, sedangkan pdb lebih tepat untuk kesalahan kompleks yang sulit ditemukan karena melibatkan banyak proses atau fungsi.

6. *Sebutkan dan jelaskan lima level logging standar dalam Python. Berikan contoh pesan yang sesuai untuk setiap level dalam konteks Sistem Informasi!*

- Lima level logging standar dalam Python adalah:

Level	**Penjelasan**	**Contoh dalam Sistem Informasi**
DEBUG	Informasi diagnostik secara detail untuk membantu menemukan masalah.	Menampilkan detail query SQL pencarian mahasiswa.
INFO	Memberikan informasi bahwa suatu proses berjalan normal.	Mahasiswa berhasil melakukan login.
WARNING	Menunjukkan kondisi yang berpotensi menimbulkan masalah.	Penggunaan penyimpanan server hampir penuh.
ERROR	Menunjukkan kesalahan yang terjadi tetapi masih dapat ditangani.	Gagal mengirim email notifikasi kepada dosen.
CRITICAL	Menunjukkan kesalahan fatal yang dapat mengganggu sistem secara keseluruhan.	Koneksi database utama terputus.

Logging berguna untuk memantau aktivitas aplikasi, menemukan penyebab kesalahan, dan membantu administrator melakukan diagnosis sistem.

7. *Apa perbedaan antara unittest.TestCase.setUp dan unittest.TestCase.setUpClass? Kapan masing-masing digunakan?*

- Perbedaannya terletak pada waktu pelaksanaannya :

- setUp() dijalankan sebelum setiap test method. Digunakan untuk menyiapkan kondisi awal yang diperlukan oleh masing-masing pengujian, seperti membuat data mahasiswa untuk setiap test.
- setUpClass() dijalankan satu kali sebelum seluruh test method dalam satu kelas dijalankan. Digunakan untuk persiapan yang cukup dilakukan sekali, seperti membuat koneksi database pengujian.

  Jadi, setUp digunakan jika persiapan perlu diulang untuk setiap pengujian, sedangkan setUpClass digunakan jika persiapan cukup dilakukan satu kali untuk seluruh kelas pengujian.

8. *Jelaskan prinsip DRY dan berikan contoh pelanggaran DRY serta refactoring yang tepat untuk mengatasinya!*

- DRY (Don't Repeat Yourself) adalah prinsip dalam pemrograman yang menganjurkan agar setiap pengetahuan atau logika dalam sistem hanya memiliki satu representasi sehingga tidak terjadi pengulangan kode.

  Contoh pelanggaran DRY:

  Dalam Sistem Informasi Akademik, proses pendaftaran mahasiswa dan proses update mahasiswa sama-sama memiliki kode validasi NIM yang ditulis berulang kali.

  Hal tersebut dapat menimbulkan masalah jika aturan validasi berubah karena pengembang harus memperbaiki kode di beberapa tempat.

  Refactoring yang tepat adalah menggunakan teknik Extract Method, yaitu memindahkan kode validasi ke dalam satu fungsi khusus.

  Dengan demikian, proses pendaftaran dan update mahasiswa cukup memanggil fungsi validasi yang sama. Kode menjadi lebih sederhana, konsisten, dan mudah dipelihara.

9. *Sebutkan minimal empat code smell umum dan teknik refactoring yang sesuai untuk masing-masing!*

- Code smell merupakan tanda bahwa struktur kode memiliki masalah yang dapat membuat program sulit dibaca, diubah, dan dipelihara, meskipun belum tentu menyebabkan kesalahan fungsi.

  Berikut beberapa code smell umum:

Code Smell	Penjelasan	Teknik Refactoring
Long Method	Fungsi memiliki terlalu banyak baris dan tugas.	Extract Method, memecah fungsi menjadi beberapa fungsi kecil.
Duplicate Code	Terdapat kode yang sama di beberapa tempat.	Extract Method atau Extract Class.
Magic Number	Menggunakan angka tanpa penjelasan yang jelas.	Replace with Constant, mengganti angka dengan konstanta bernama.
Deep Nesting	Terlalu banyak percabangan if bertingkat.	Guard Clause atau Extract Method.
Dead Code	Kode yang tidak pernah digunakan atau dijalankan.	Remove Dead Code.
God Class	Kelas memiliki terlalu banyak tanggung jawab.	Extract Class, memisahkan tanggung jawab ke kelas lain.

Refactoring dilakukan agar kode lebih mudah dipahami, mengurangi risiko bug, dan memudahkan pengembangan sistem.

10. *Apa peran static analysis tools (pylint, flake8, black, mypy) dalam proses code review? Bagaimana integrasi ke CI/CD pipeline meningkatkan efektivitas review?*

- Static analysis tools merupakan alat untuk memeriksa kode program tanpa harus menjalankannya. Alat ini membantu mendeteksi potensi kesalahan, pelanggaran aturan penulisan, dan masalah kualitas kode secara otomatis sebelum dilakukan code review oleh manusia. Fungsi masing-masing tools:

Tools	Fungsi

Pylint	Melakukan pemeriksaan kode secara menyeluruh, mendeteksi potensi kesalahan, dan memberikan penilaian kualitas kode.
Flake8	Memeriksa kesesuaian kode dengan standar PEP 8, kompleksitas kode, dan import.
Black	Mengatur format kode secara otomatis agar konsisten dan mudah dibaca.
Mypy	Melakukan pemeriksaan tipe data (type checking) untuk menemukan ketidaksesuaian tipe data.

Integrasi dengan CI/CD pipeline:

Integrasi static analysis tools ke dalam CI/CD pipeline memungkinkan pemeriksaan kode dilakukan secara otomatis setiap kali pengembang melakukan perubahan atau mengirim kode ke repository.

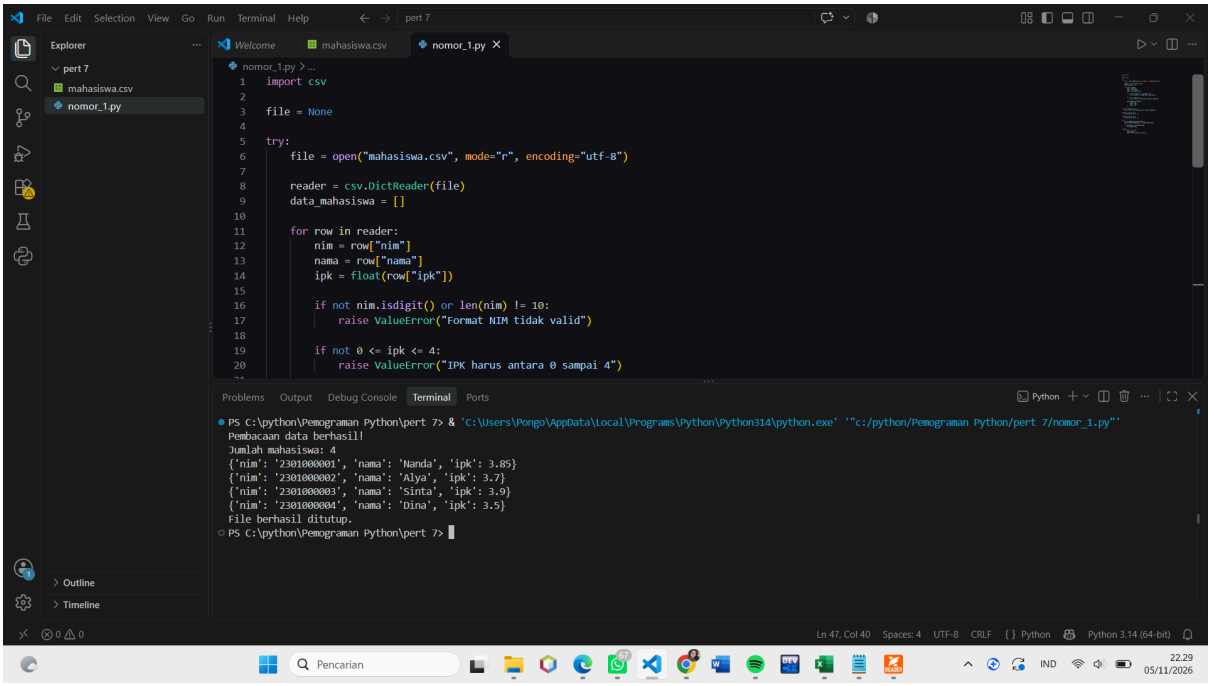
Manfaatnya:

1. Mendeteksi kesalahan lebih awal sebelum kode digabungkan ke branch utama.
2. Menjaga konsistensi format dan standar penulisan kode.
3. Mengurangi pekerjaan pemeriksaan manual oleh reviewer.
4. Membantu mencegah kode yang bermasalah masuk ke codebase utama.
5. Membuat proses code review lebih efektif karena reviewer dapat lebih fokus pada logika bisnis, keamanan, dan arsitektur program.

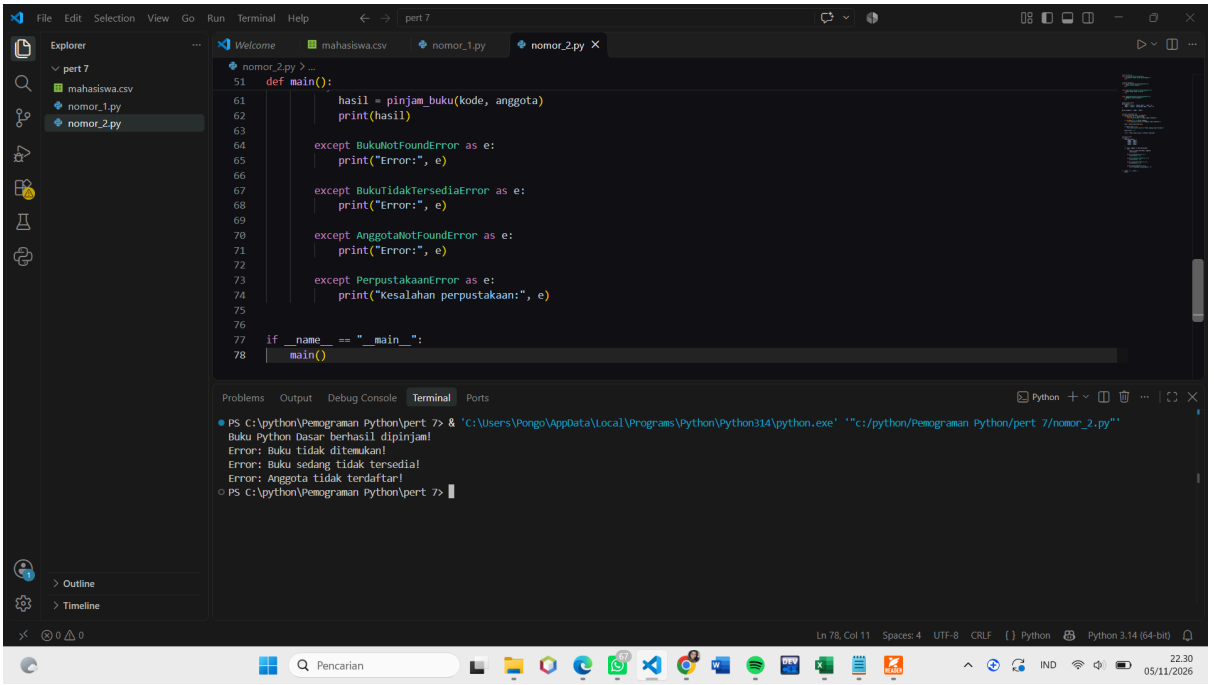
Dengan demikian, kombinasi static analysis tools dan CI/CD membantu meningkatkan kualitas kode serta mempercepat proses code review.

B. SOAL PRAKTIK

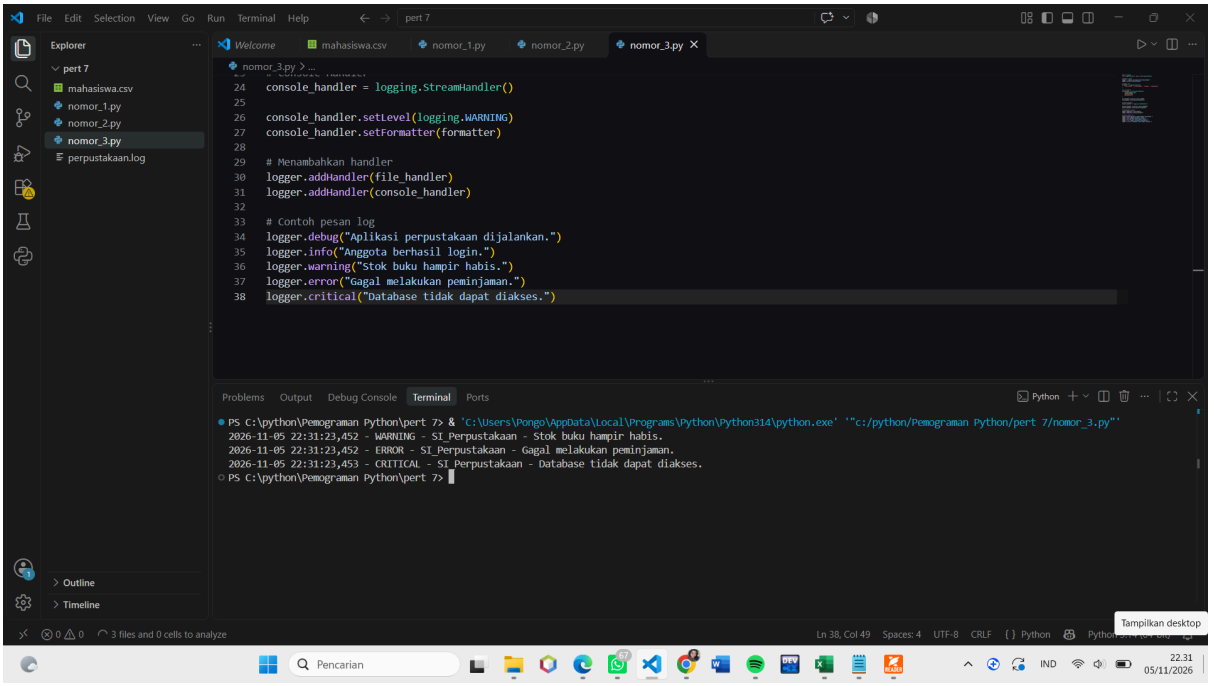
1. Buatlah program Python yang membaca data mahasiswa dari file CSV dengan penanganan exception lengkap: FileNotFoundError, csv.Error, dan ValueError. Gunakan blok finally untuk memastikan file ditutup dan blok else untuk menampilkan ringkasan data jika pembacaan berhasil.

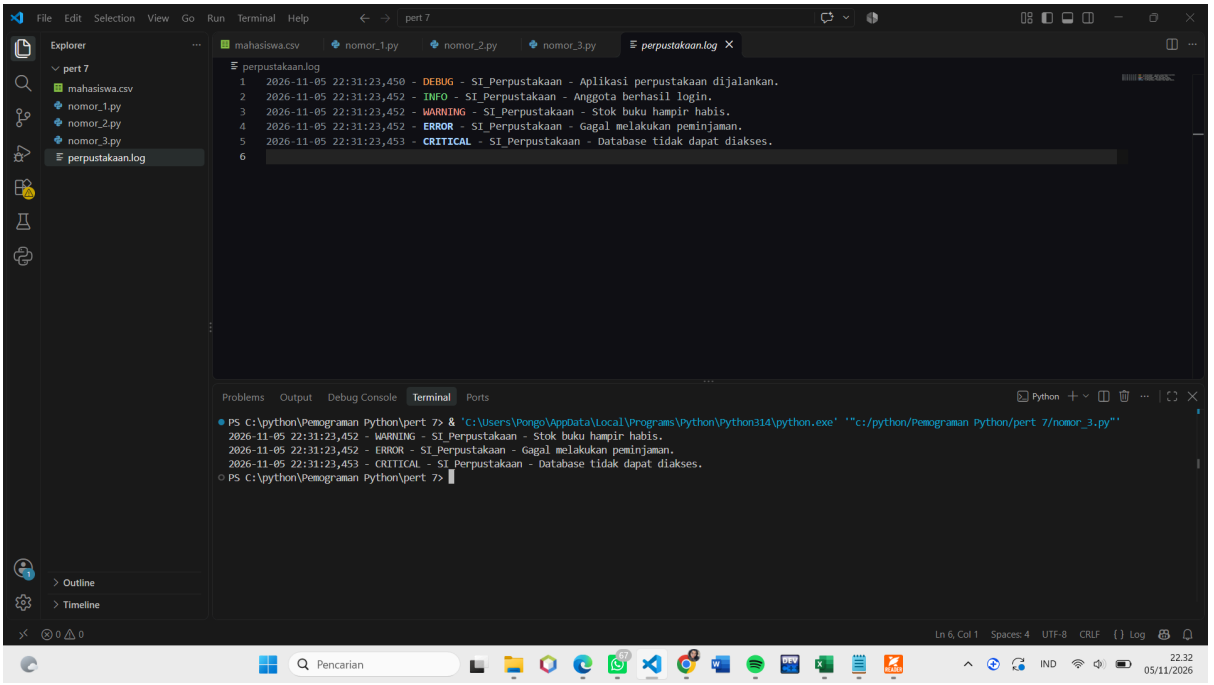


2. Buatlah hierarki custom exception untuk Sistem Informasi Perpustakaan dengan minimal: PerpustakaanError sebagai base, BukuNotFoundError, BukuTidakTersediaError, and AnggotaNotFoundError. Implementasikan fungsi pinjam_buku() yang menggunakan exception tersebut dan tulis program utama yang mendemonstrasikan penanganan setiap jenis exception.

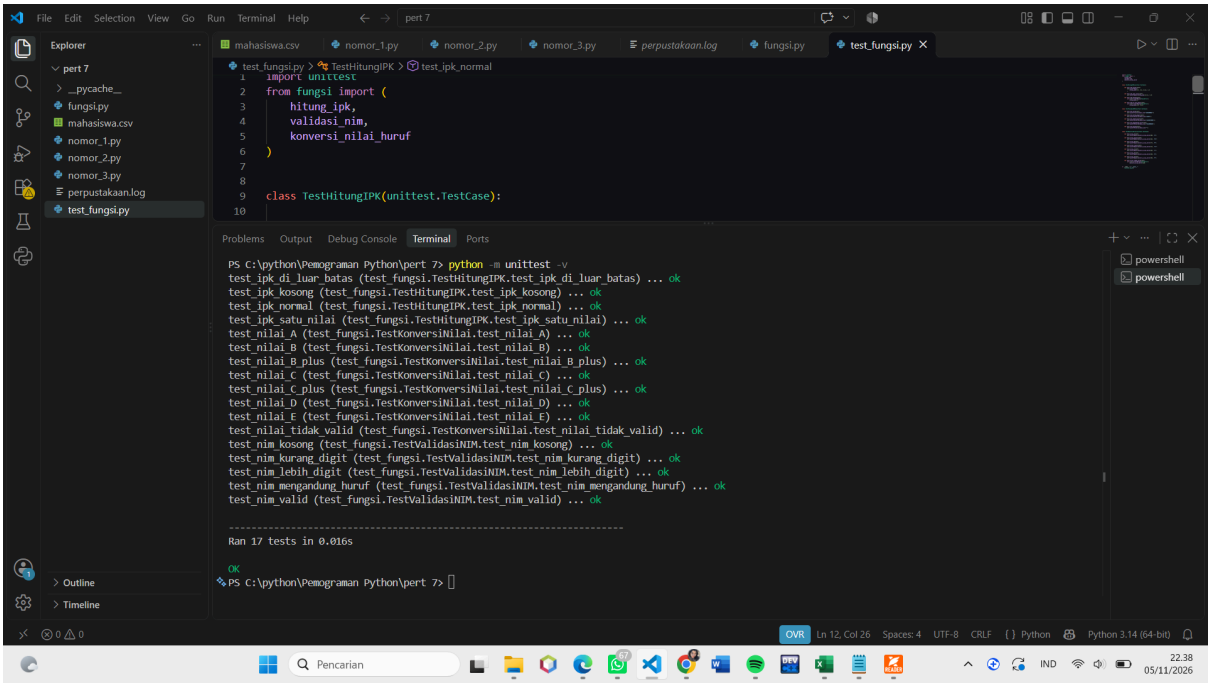


3. Konfigurasikan modul logging Python untuk aplikasi SI dengan: (a) File handler dengan RotatingFileHandler untuk log DEBUG ke atas, (b) Console handler untuk log WARNING ke atas, (c) Formatter yang menyertakan timestamp, level, nama logger, dan pesan. Tulis beberapa pesan log pada berbagai level dan verifikasi output pada kedua handler.

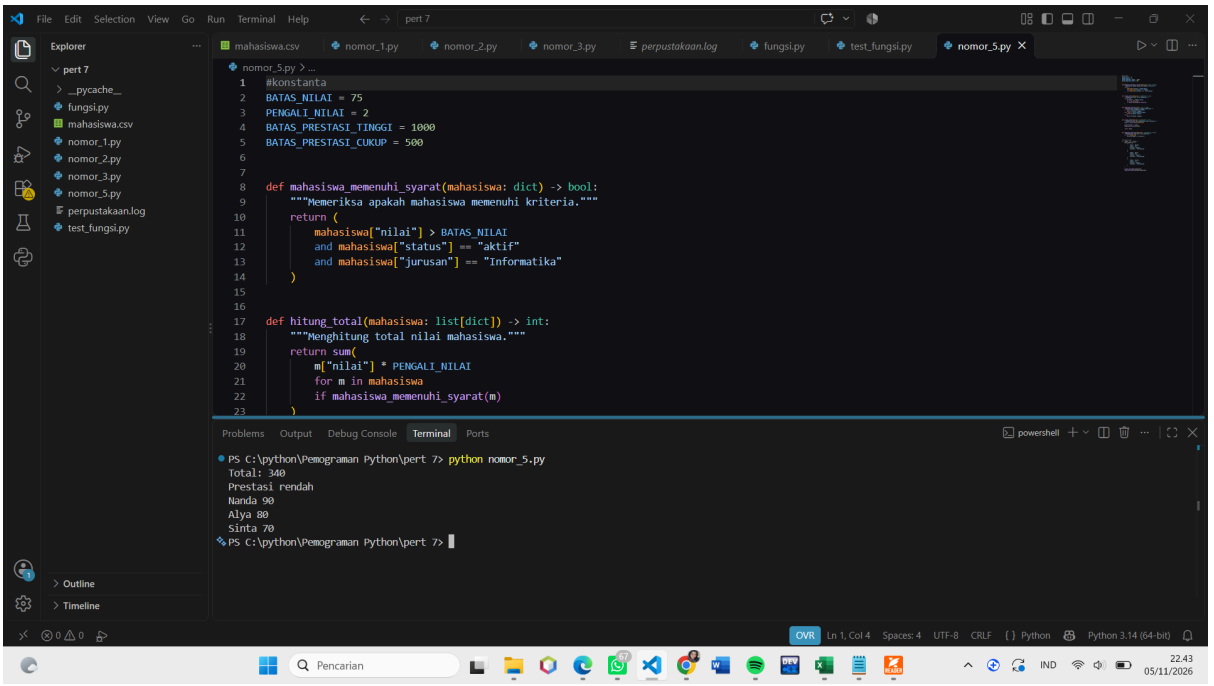




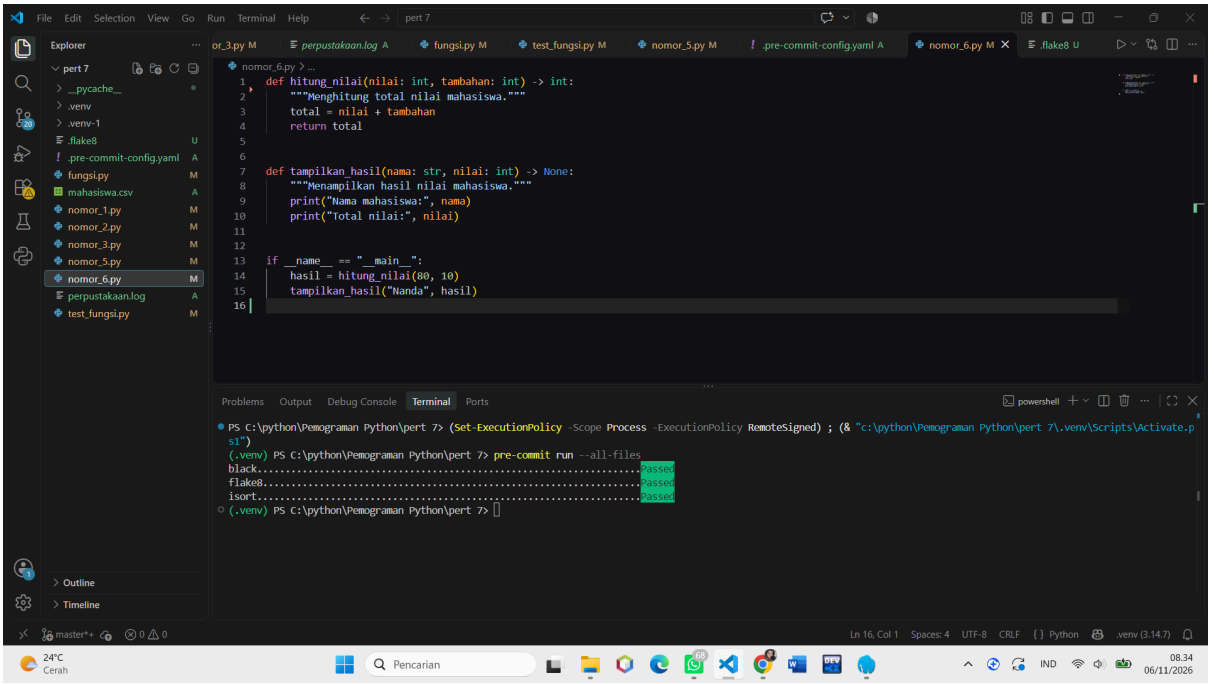
4. Tulis unit test lengkap menggunakan unittest untuk fungsi berikut: (a) hitung_ipk() yang menghitung IPK dari list nilai, (b) validasi_nim() yang memvalidasi format NIM 10 digit, (c) konversi_nilai_huruf() yang mengkonversi angka ke huruf (A, B+, B, dst). Sertakan test untuk happy path, edge case, dan input tidak valid.

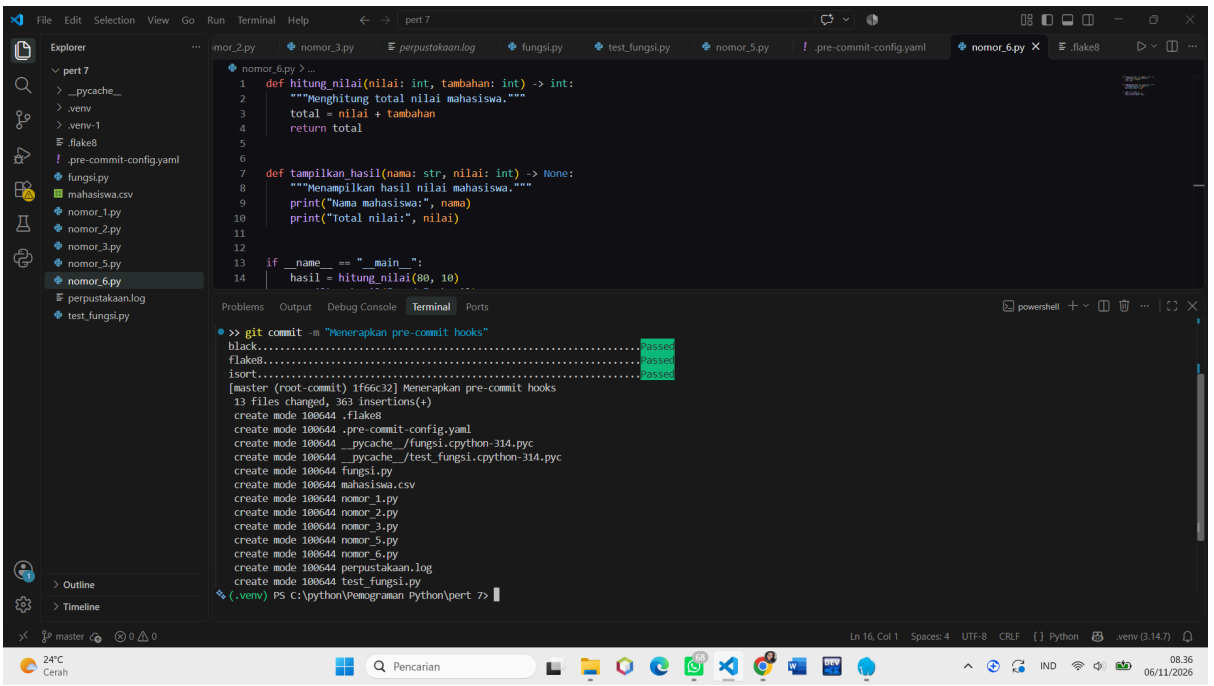


5. Diberikan kode yang mengandung minimal lima code smell (magic number, duplikasi, long method, deep nesting, dead code). Identifikasi setiap code smell, jelaskan masalahnya, dan lakukan refactoring untuk menghilangkannya. Tambahkan docstring dan type hints sesuai PEP 8.



6. Buatlah konfigurasi pre-commit hooks yang menjalankan black (autoformat), flake8 (linting), dan isort (import sorting) secara otomatis sebelum setiap commit. Tulis file .pre-commit-config.yaml dan demonstrasikan bagaimana hooks ini mencegah kode yang tidak memenuhi standar masuk ke repository


def hitung_nilai(nilai: int, tambahan: int) -> int:
"""Menghitung total nilai mahasiswa."""
total = nilai + tambahan
return total
def tampilkan_hasil(nama: str, nilai: int) -> None:
"""Menampilkan hasil nilai mahasiswa."""
print("Nama mahasiswa:", nama)
print("Total nilai:", nilai)
if __name__ == "__main__":
hasil = hitung_nilai(80, 10)
tampilkan_hasil("Nanda", hasil)
PS C:\python\Pemograman Python\pert 7> pre-commit run --all-files
black.....Passed
flake8.....Passed
isort.....Passed
(.venv) PS C:\python\Pemograman Python\pert 7>


>> git commit -m "Menerapkan pre-commit hooks"
black.....Passed
flake8.....Passed
isort.....Passed
[master (root-commit) 1f66c32] Menerapkan pre-commit hooks
13 files changed, 363 insertions(+)
create mode 100644 .flake8
create mode 100644 .pre-commit-config.yaml
create mode 100644 __pycache__/fungsi.cpython-314.pyc
create mode 100644 __pycache__/test_fungsi.cpython-314.pyc
create mode 100644 fungsi.py
create mode 100644 mahasiswa.csv
create mode 100644 nomor_1.py
create mode 100644 nomor_2.py
create mode 100644 nomor_3.py
create mode 100644 nomor_5.py
create mode 100644 nomor_6.py
create mode 100644 perpustakaan.log
create mode 100644 test_fungsi.py
(.venv) PS C:\python\Pemograman Python\pert 7>